

Kontrola chodu programu

Základy programování

Matouš Cejnek

U12110, FS, ČVUT v Praze

Co si dnes odnesete?

- Jak přesně podmínky a cykly určují **další vykonaný krok**?
- Jak skládat logické výrazy a využít **krátké vyhodnocování**?
- Kdy volit for, kdy while a jak poznat, že cyklus skončí?
- Jak číst break, continue a else u cyklu?
- Kdy pomohou ternární výraz a **list comprehension**?

Na co navazujeme?

- Z návrhu algoritmů známe sekvenci, podmínky, cykly a diagramy.
- Z datových typů víme, že výraz má hodnotu a typ.
- Už jsme psali `if`, `for` a `while`.

Dnešní otázka

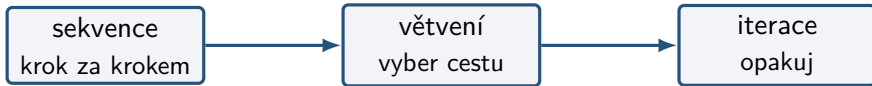
Dokážeme z kódu spolehlivě určit, **co se vykoná, kolikrát a proč?**

Rychlé opakování: sledujte stav

```
total = 0
for number in [3, -1, 4]:
    if number > 0:
        total += number
print(total)
```

- Které řádky se vykonají opakovaně?
- Jaké hodnoty má `number` a `total` po každém průchodu?
- Výsledek nevzniká „najednou“: program mění **stav** krok za krokem.

Tři stavební kameny řízení toku



- Konstrukce lze **vnořovat**: podmínka v cyklu, cyklus v podmínce.
- Odsazení v Pythonu určuje, které příkazy tvoří jeden blok.

Podmínka rozhoduje podle pravdivostní hodnoty

Podmínka

Výraz, jehož výsledek Python vyhodnotí jako True, nebo False.

- Porovnání: `x < 0`, `name == "Eva"`, `x in values`.
- Logické operátory: `and`, `or`, `not`.
- Podmínka se vyhodnotí právě ve chvíli, kdy k ní tok programu dorazí.

Pravdivost: nejen bool

Nepravdivé hodnoty

- False, None
- číselná nula
- "", [], {}

Čitelný zápis

`if values:` přímo říká „pokud kolekce není prázdná“. Pravdivost však není totéž co rovnost s `True`.

Pravdivé hodnoty

- většina ostatních hodnot
- neprázdný text či kolekce
- také záporné číslo

Porovnání lze řetězit

```
if 0 <= temperature <= 100:  
    print("V kapalnem intervalu vody")
```

- Význam odpovídá `0 <= temperature and temperature <= 100`.
- Matematický zápis bývá čitelnější a prostřední výraz se vyhodnotí jednou.
- Nezaměňujte `=` (přiřazení) a `==` (porovnání).

Jak se skládají logické podmínky?

A	B	A and B	A or B
F	F	F	F
F	T	F	T
T	F	F	T
T	T	T	T

- and: musejí platit obě části.
- or: stačí alespoň jedna část.
- not: obrátí pravdivostní hodnotu.

Krátké vyhodnocování chrání další výraz

```
if denominator != 0 and numerator / denominator > 2:  
    print("Podíl je větší než 2")
```

- U `and` se druhá část nevyhodnotí, pokud první nestačí splnit.
- U `or` se druhá část nevyhodnotí, pokud první už stačí.
- Pořadí podmínek proto ovlivňuje nejen rychlost, ale někdy i bezpečnost.

Jedna z více větví

```
if score >= 75:  
    grade = "B"  
elif score >= 60:  
    grade = "C"  
else:  
    grade = "F"
```

- Podmínky se zkoušejí shora dolů; provede se **první platná** větev.
- Pořadí musí jít od specifitějšího případu k obecnějšímu. Dvě samostatná `if` jsou naopak dvě nezávislé otázky.

Podmíněný výraz: hodnota podle podmínky

```
label = "sude" if number % 2 == 0 else "liche"
```

- Celý zápis je **výraz**: vytvoří jednu ze dvou hodnot.
- Hodí se pro jednoduchou volbu při přiřazení nebo výpisu.
- Pro více kroků či složitou logiku je čitelnější běžné `if`.

Kontrolní otázka: co program vypíše?

```
x = 0
if x and 10 / x > 1:
    print("A")
elif x == 0:
    print("B")
else:
    print("C")
```

Zdůvodněte bez spuštění

Sledujte pravdivost hodnoty x , krátké vyhodnocování a pořadí větví.

Iterace opakuje blok příkazů

- Jedno vykonání těla cyklu je **iterace**.
- Cyklus `for` získává postupně hodnoty z **iterovatelného objektu**.
- Cyklus `while` znovu testuje podmínku.
- Po každé iteraci se musí změnit hodnota, pozice nebo jiný stav, jinak nemusíme postupovat ke konci.

Pythoní for prochází hodnoty

```
for color in ["red", "green", "blue"]:  
    print(color)
```

- Proměnná `color` postupně odkazuje na každý prvek.
- Nepotřebujeme ručně spravovat index ani podmínku ukončení.
- Stejný princip funguje pro seznam, text, n-tici, množinu i slovník.

range: posloupnost celých čísel

```
range(5)           # 0, 1, 2, 3, 4  
range(2, 6)        # 2, 3, 4, 5  
range(10, 3, -2)   # 10, 8, 6, 4
```

- Pravá mez se **nezahrnuje**.
- Třetí argument je krok; záporný krok umožní sestup.
- range použijeme, když opravdu potřebujeme čísla nebo počet opakování.

Když potřebujeme prvek i jeho pozici

```
names = ["Ada", "Grace", "Guido"]  
  
for position, name in enumerate(names, start=1):  
    print(position, name)
```

- `enumerate` vytváří dvojice (index, hodnota).
- Je čitelnější než ruční zvyšování počítadla.
- Parametr `start` mění počáteční číslo, ne indexování seznamu.

Když procházíme dvě kolekce souběžně

```
names = ["Ada", "Grace", "Guido"]  
scores = [95, 91, 88]  
  
for name, score in zip(names, scores):  
    print(name, score)
```

- zip vytváří dvojice odpovídajících prvků.
- Běžný zip skončí s nejkratší kolekcí.
- Pokud délky musejí být stejné, ověřte tento předpoklad.

while: opakování řízené stavem

```
remaining = 23
steps = 0

while remaining > 0:
    remaining -= 5
    steps += 1
```

- Podmínka se testuje **před** každou iterací.
- Tělo se nemusí vykonat ani jednou.
- while se hodí, když předem neznáme počet iterací.

Tři otázky ke správnosti cyklu

- 1 **Inicializace:** platí před prvním průchodem potřebný stav?
- 2 **Pokrok:** přibližuje každá iterace program ke konci?
- 3 **Výsledek:** co víme, když podmínka přestane platit?

Invarianta cyklu

Tvrzení, které platí před každou iterací a pomáhá vysvětlit správnost výsledku.

Příklad invarianty: průběžný součet

```
total = 0
for number in numbers:
    total += number
```

- Před každou iterací je `total` součtem už zpracovaných prvků.
- Přidáním aktuálního prvku tvrzení zachováme pro další iteraci.
- Po skončení byly zpracovány všechny prvky, proto je `total` jejich součet.

Jak vznikne nekonečný cyklus?

```
remaining = 10
while remaining > 0:
    print(remaining)
    # remaining se nemění
```

Diagnóza

Podmínka závisí na `remaining`, ale tělo tuto hodnotu neposouvá k nepravdě.

- Někdy je nekonečný cyklus záměrný, například u serveru.
- Musí pak existovat jiný kontrolovaný způsob ukončení.

break a continue

```
for number in numbers:
    if number < 0:
        continue
    if number == target:
        break
    print(number)
```

- continue: přeskoč zbytek **této iterace**.
- break: opust **nejbližší cyklus**.
- Oba příkazy mění běžný tok; používejte je tam, kde záměr zpřehlední.

else u cyklu: nebyl proveden break

```
for number in numbers:
    if number == target:
        print("Nalezeno")
        break
else:
    print("Nenalezeno")
```

- else proběhne po normálním dokončení cyklu.
- Neproběhne, pokud cyklus ukončil break.
- Neznamená „cyklus neměl žádnou iteraci“.

Volba cyklu vyjadřuje záměr

Použijte `for`

- pro každý prvek,
- pevný počet opakování,
- známý rozsah hodnot.

Použijte `while`

- dokud platí podmínka,
- čekání na správný vstup,
- počet kroků předem neznáme.

Vnoření mění počet provedení

```
for row in table:           # n radku
    for value in row:       # m hodnot v radku
        print(value)
```

- Tělo vnitřního cyklu se vykoná přibližně $n \cdot m$ -krát.
- Dva cykly **za sebou** by se naopak sčítaly.
- Struktura toku proto přímo souvisí s časovou složitostí.

Historie: od skoku k pojmenovaným strukturám

- Procesor umí změnit pořadí instrukcí pomocí podmíněných a nepodmíněných skoků.
- Příkaz goto přenesse tok programu na označené místo.
- Libovolné skoky jsou mocné, ale dovolují mnoho cest vstupu do stejné části programu.
- Výsledek může připomínat „špagetový kód“: tok se obtížně sleduje a zdůvodňuje.

1966 a 1968: strukturované programování

- Böhm a Jacopini (1966) zkoumali převod obecných diagramů toku do strukturované podoby.
- Praktická myšlenka: sekvence, výběr a opakování stačí k vyjádření algoritmů.
- Dijkstra (1968) kritizoval nekontrolované goto: ztěžuje vztah mezi textem programu a jeho průběhem.
- Cílem není zákaz skoků v procesoru, ale **čitelný tok ve zdrojovém kódu**.
- `break` a `continue` proto mají omezený, z kódu zřejmý cíl.

C. Böhm, G. Jacopini, 1966, DOI: 10.1145/355592.365646; E. W. Dijkstra, 1968, DOI: 10.1145/362929.362947.

Běžný vzor: vytvoření nového seznamu

```
squares = []  
for number in numbers:  
    squares.append(number ** 2)
```

- Začneme prázdným výsledkem.
- Pro každý vstup vypočteme jednu hodnotu.
- Hodnotu přidáme do nového seznamu.

List comprehension: stejný datový tok stručně

```
squares = [number ** 2 for number in numbers]
```

[výraz for prvek in kolekce]

- Výsledkem je nový seznam.
- Pořadí čtení: „druhá mocnina pro každé číslo v číslech“.
- Je to vhodné pro jednoduchou transformaci, ne pro libovolný blok příkazů.

Filtrování: if za cyklem

```
positive_squares = [  
    number ** 2  
    for number in numbers  
    if number > 0  
]
```

- Koncové if rozhoduje, **zda prvek zahrnout**.
- Výraz na začátku říká, **jakou hodnotu vytvořit**.
- Víceřádkový zápis je stále jeden výraz.

Volba hodnoty: if před cyklem

```
labels = [  
    "sude" if number % 2 == 0 else "liche"  
    for number in numbers  
]
```

- Ternární výraz vytvoří hodnotu pro **každý** prvek.
- Koncové if by některé prvky naopak vyřadilo.
- Poloha if tedy mění význam, nejde jen o styl zápisu.

Kdy zůstat u obyčejného cyklu?

- Tělo provádí více různých akcí nebo mění více proměnných.
- Potřebujeme `break`, `continue` nebo podrobný výpis průběhu.
- Comprehension má několik vnořených cyklů či podmínek a musí se luštit.
- Zkrácení počtu řádků samo o sobě není cílem.

Pravidlo čitelnosti

Kompaktní zápis použijte, pokud jedním pohledem sděluje jednu transformaci.

Závěrečné ověření: tři otázky

```
result = [x * 2 for x in values if x]
```

- 1 Které hodnoty podmínka `if x` vyřadí?
- 2 V jakém pořadí se vyhodnotí cyklus, podmínka a výraz `x * 2`?
- 3 Kdy by byl běžný cyklus čitelnější?

Co si zapamatovat?

- Tok programu skládáme ze sekvence, větvení a iterace.
- Pořadí podmínek a krátké vyhodnocování ovlivňují vykonané operace.
- U cyklu sledujte inicializaci, invariantní stav a pokrok ke konci.
- `for` prochází hodnoty; `while` opakuje podle stavu.
- `break`, `continue` a `else` přesně mění běžný tok cyklu.
- Ternární výraz a comprehension jsou dobré tehdy, když zpřehlední jednoduchý záměr.

Otázky?